



Programação Avançada 2025/26
2 Introdução aos Testes Unitários (Coleções)

@Patricia Macedo & Bruno Silva

Sumário Aula 2

- Teste unitário
- JUnit
- Testes Unitários para testar ADTs do tipo coleção.

Testes Unitários, o que são?

- Teste à menor unidade de código possível.
- A menor unidade de código em JAVA é o método.
- Um teste unitário é implementado através de um ou mais métodos que testam se o comportamento esperado de um determinado método se cumpre.
- Existem frameworks específicas , para cada tipo de Linguagem, para criar testes unitários

Testes Unitários, para que servem?

- Os testes de software servem para garantir a qualidade do mesmo.
- Para garantir a qualidade de um produto temos que verificar se esse cumpre as funcionalidades especificadas.
- Mas porque não fazemos só testes quando o produto de software está finalizado ?

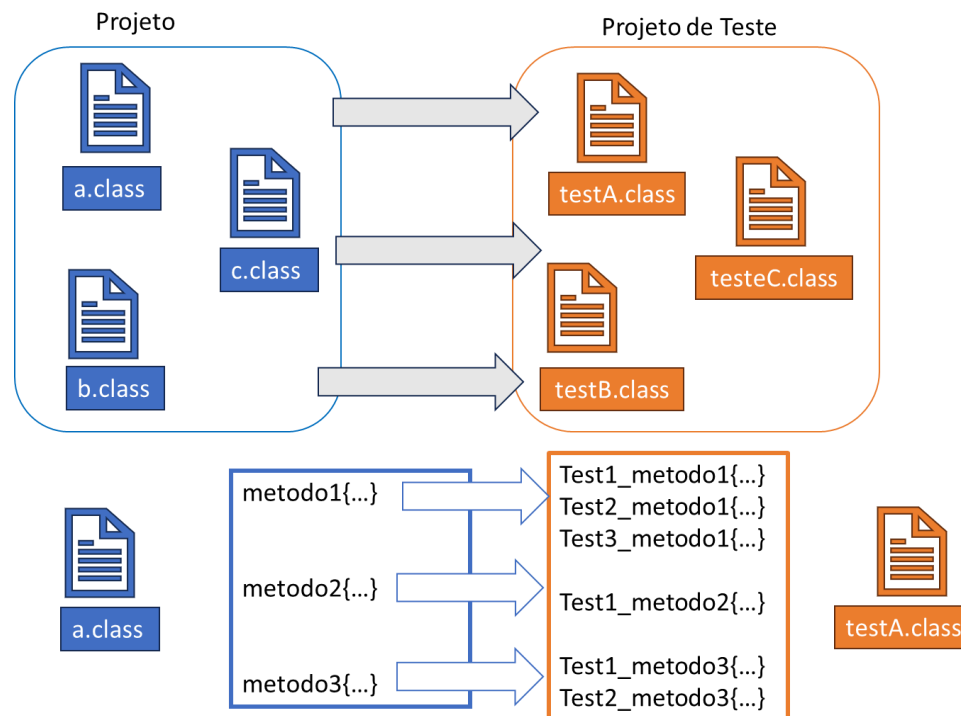
Pirâmide de Testes

- Quanto mais cedo detetarmos um erro, mais fácil (e consequentemente mais barato) é corrigi-lo.
Os testes Unitários são o primeiro níveis de testes.



Testes Unitários - Como funcionam ?

- Os testes unitários vão servir para testar unidades de código - métodos/funções
- Um teste unitários é em si um método que testa o funcionamento de outro método.



Como criar um Teste Unitário usando o JUnit

Testar o método 'decimal2Binary'

```
/**
 * @param decimal- positive integer
 * @return a String with integer convert to binary
 * @IllegalArgumentException if decimal is negative
 */
public static String decimal2Binary(int decimal)
    throws IllegalArgumentException{
    ...
}
```

Como criar um Teste Unitário usando o JUnit

1. Pensar em "Test Cases" (Casos de Teste)
2. Criar um ou mais testes (métodos) para implementar todos os testCases

Test Cases

- 1 - Testar que o resultado da conversão é correto (testar para diversos valores por exemplo: para os valores <0>, <5> e <256>)
- 2 - Testar o lançamento correto da exceção

Como criar um Teste Unitário usando o JUnit

```
class DecimalToBinaryTest {  
  
    @Test  
    void decimal2Binary() {  
        assertEquals("0",DecimalToBinary.decimal2Binary(0));  
        assertEquals("101",DecimalToBinary.decimal2Binary(5));  
        assertEquals("100000000",DecimalToBinary.decimal2Binary(256));  
    }  
  
    @Test  
    void decimal2Binary_ExceptionWhenNegative() {  
        assertThrows(IllegalArgumentException.class, () -> {  
            DecimalToBinary.decimal2Binary(-5);  
        });  
        assertThrows(IllegalArgumentException.class, () -> {  
            DecimalToBinary.decimal2Binary(-1);  
        });  
    }  
}
```

E como testar coleções ?

As coleções testam-se da mesma forma que as restantes classes:

- Desenhar os *TestCases*
- Implementar um método para cada *TestCase*

Nota – O objetivo dos testes a uma classe é garantir que esta se comporta como o especificado.

- Cada ADT do tipo coleção (stack,queue,list,set), tem um conjunto de características específicas.
- Os TesteCase tem que ser desenhados de forma a testar se essas características são cumpridas (tipo de acesso, repetição de elementos , emissão correta de erros etc....).

Exemplo: Testar a implementação do ADT Stack

- Para cada uma das operações especificadas na interface Stack, planear um ou mais *Test Case* de forma a testar se cada uma das operações se comporta como é suposto.

```
public interface Stack<T> {  
    public void push(T element) throws FullStackException;  
    public T pop() throws EmptyStackException;  
    public T peek() throws EmptyStackException;  
  
    public int size();  
    public boolean isEmpty();  
    public void clear();  
}
```

Exemplo: Testar a implementação do ADT Stack

Operação	TestCase	Estratégia
size	size_isCorrect_AfterPushPop()	• Introduzir n elementos e testar se o tamanho aumenta de 0 a n. • Remover n elementos e testar se o tamanho diminui de n a 0.
peek	peek_isCorrect_AfterPushPop()	• Introduzir n elementos e após cada element verificar se o elemento do topo é o ultimo elemento introduzido. • Retirar n-1 elementos e verificar se após a sua retirada o elemento do topo é o expectavel.
empty	isEmpty_iscorrect_AfterPushPop	• Verificar se o stack inicial está vazio (se a operação devolve true) • Introduzir n elementos • Verificar se o stack não está vazio (se a operação devolve false) • Retirar todos n elementos • Verificar se o stack inicial está vazio (se a operação devolve true)

Exemplo: Testar a implementação do ADT Stack -(cont)

Operações	TestCase	Estratégia
peek	peekExceptionWhenEmpty()	• Executar a operação de peek, sobre uma stack vazia e verificar que a exceção de Empty é lançada
push	pushExceptionWhenFull()	• Nota: Só é possível executar este teste, se for possível definir uma dimensão máxima para a pilha. • Introduzir o número de elementos na pilha igual à quantidade máxima • Introduzir um element extra e verificar se lança a exceção de Full.
pop	popExceptionWhenEmpty()	• Executar a operação de pop, sobre uma stack vazia e verificar que a exceção de Empty é lançada

Exemplo: class StackArrayListTest

- Teste

```
class StackArrayListTest {  
    private Stack<Integer> stack;  
  
    @BeforeEach  
    public void setUp() {  
        stack = new StackArrayList<>();  
    }  
    ....  
    //test cases  
}
```

Exemplo: Testar a operação size

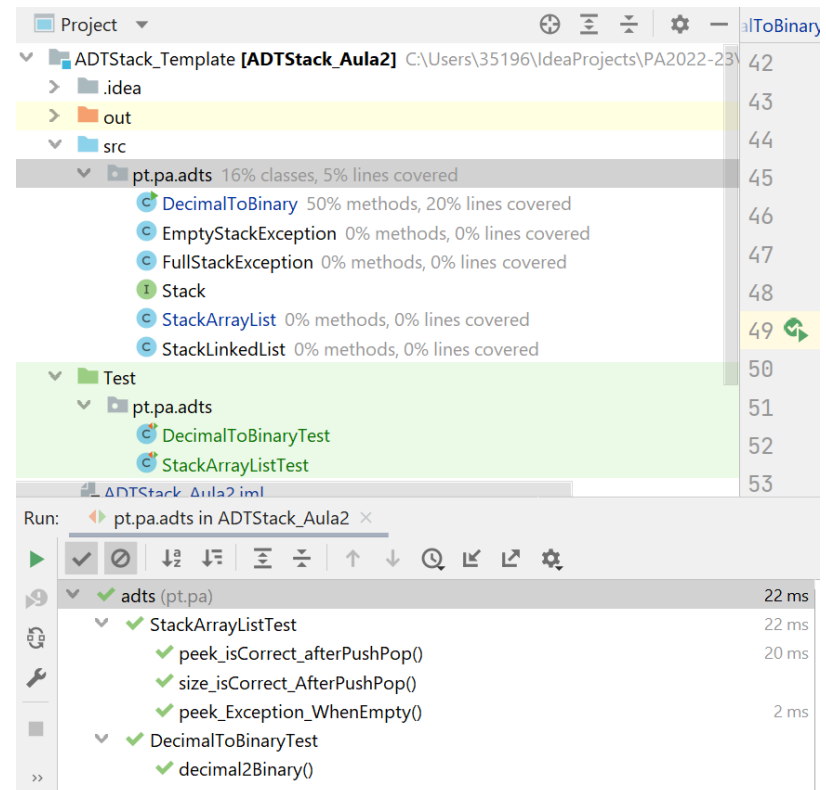
```
@Test
    public void sizeIsCorrectAfterPushPop() {
        assertEquals(0, stack.size(), "size is not correct!");
        stack.push(1);
        assertEquals(1, stack.size(), "size is not correct!");
        stack.push(2);
        assertEquals(2, stack.size(), "size is not correct!");
        stack.push(3);
        assertEquals(3, stack.size(), "size is not correct!");
        stack.pop();
        assertEquals(2, stack.size(), "size is not correct!");
        stack.pop();
        stack.pop();
        assertEquals(0, stack.size(), "size is not correct!");
    }
```

Exemplo: Testar o lançamento correto de exceção

```
@Test
    public void peekExceptionWhenEmpty() {
        assertThrows(EmptyStackException.class, () -> {
            stack.peek();
        })
    }
```


Automatizar os testes em JUnit

- Classes a serem testada.
- Classes de teste.
- Resultado dos testes.



Exercícios práticos

- Complete os testes para o ADT Stack implementado na última aula.

https://github.com/estsetubal-pa/ADTStack_Template

- Ver JUnit no IntelliJ IDEA

<https://www.jetbrains.com/help/idea/junit.html>